VERITAS UNIVERSITY, ABUJA

Department of Software Engineering — SEN307: Database Systems

Lab 2 Practical Solution: Design & Normalization

Student Name: Joseph Ijanu Skari **Matric No:** VUG/SEN/24/11477 **Academic Session:** 2026/2027

PART 1: ENTITIES, ATTRIBUTES, AND RELATIONSHIPS

Step 1: Entity-Attribute Identification Table

Below is the complete entity list derived from the course-registration scenario, specifying primary keys, foreign keys, and attributes as defined in the solution.

Entity	Attributes	Primary Key (PK)	Foreign Key(s) (FK)
Department	dept_id, dept_name	dept_id	*None*
Student	matric_no, name, dept_id	matric_no	*dept_id* (references Department)
Lecturer	lecturer_id, lecturer_name, dept_id	lecturer_id	*dept_id* (references Department)
Course	course_code, course_title	course_code	*None*
Enrolment	matric_no, course_code, session	matric_no, course_code	*matric_no*, *course_code*
CourseAndLecturer	course_code, lecturer_id, session	course_code, lecturer_id, session	*course_code*, *lecturer_id*

Step 2: Cardinality Analysis & Junction Entities

- **Student** `1` — `N` **Department:** A student belongs to exactly **one** department, whereas a department can host **many** students (**1:N Relationship**). The primary key `dept_id` is placed as a foreign key in the `Student` entity.
- **Student** `M` — `N` **Course:** One student can register for **many** courses, and one course can be taken by **many** students (**M:N Relationship**). This requires a junction table (`Enrolment`).
- **Lecturer** `M` — `N` **Course:** A course may have **many** lecturers across different sessions, and a lecturer can teach **many** courses (**M:N Relationship**). This requires a second junction table (`Course_Lecturer`).

> **Junction Entities Identified:** Relational engines cannot natively model direct M:N links. We resolve them using two bridge tables: `Enrolment` (linking Student and Course) and `Course_Lecturer` (linking Lecturer and Course per Session).

PART 2: STEP-BY-STEP NORMALIZATION (1NF TO 3NF)

Unnormalized Form (UNF) Data Representation

The original registration sheet contains multi-valued repeating groups in several attributes:

matric_no	student_name	student_dept	course_codes	course_titles	lecturers	lecturer_depts	session
20231234	Ada Obi	Software Eng.	SEN307, SEN301	Database Systems, Software Architecture	Dr A. Okafor, Dr M. Bello	Software Eng., Software Eng.	2026/2027
20231235	John Musa	Computer Sci.	SEN307, COS301	Database Systems, Operating Systems	Dr A. Okafor, Mrs F. Eze	Software Eng., Computer Sci.	2026/2027
20231236	Grace Okon	Software Eng.	SEN301	Software Architecture	Dr M. Bello	Software Eng.	2026/2027

Step 1: First Normal Form (1NF)

To achieve 1NF, multi-valued cells are atomicized by expanding repeating values so that each record represents a single student-course-lecturer instance.

matric_no	student_name	student_dept	course_code	course_title	lecturer_name	lecturer_dept	session
20231234	Ada Obi	Software Eng.	SEN307	Database Systems	Dr A. Okafor	Software Eng.	2026/2027
20231234	Ada Obi	Software Eng.	SEN301	Software Architecture	Dr M. Bello	Software Eng.	2026/2027
20231235	John Musa	Computer Sci.	SEN307	Database Systems	Dr A. Okafor	Software Eng.	2026/2027
20231235	John Musa	Computer Sci.	COS301	Operating Systems	Mrs F. Eze	Computer Sci.	2026/2027
20231236	Grace Okon	Software Eng.	SEN301	Software Architecture	Dr M. Bello	Software Eng.	2026/2027

> **1NF Explanation:** We eliminated multi-valued cells so that every cell contains a single atomic value, and defined the composite primary key as `(matric_no, course_code, lecturer_name)`.

Step 2: Second Normal Form (2NF)

We identify and remove partial dependencies where non-key attributes depend on only a subset of the composite primary key.

- `matric_no` $\rightarrow$ `student_name`, `student_dept` (Partial dependency on student identity).
- `course_code` $\rightarrow$ `course_title` (Partial dependency on course code).
- `lecturer_name` $\rightarrow$ `lecturer_dept` (Partial dependency on lecturer identity).

Decomposed 2NF Tables:

Table 1: Student_2NF (Key: **matric_no**)

matric_no (PK)	student_name	student_dept
20231234	Ada Obi	Software Eng.
20231235	John Musa	Computer Sci.
20231236	Grace Okon	Software Eng.

Table 2: Course_2NF (Key: **course_code**)

course_code (PK)	course_title
SEN307	Database Systems
SEN301	Software Architecture
COS301	Operating Systems

Table 3: Lecturer_2NF (Key: **lecturer_name**)

lecturer_name (PK)	lecturer_dept
Dr A. Okafor	Software Eng.
Dr M. Bello	Software Eng.
Mrs F. Eze	Computer Sci.

Step 3: Third Normal Form (3NF)

To reach 3NF, we must eliminate transitive dependencies where a non-key attribute depends on another non-key attribute. Specifically, `student_dept` and `lecturer_dept` represent department entities. If a department name changes, updating it in multiple student/lecturer rows causes update anomalies.

> **3NF Transformation Rule:** Separate department details into a dedicated `Department` table and reference `dept_id` as a Foreign Key across `Student` and `Lecturer` entities. Also introduce surrogate integer Primary Keys for optimal indexing.

Step 4: Final 3NF Relational Schema

Below is the complete, normalized schema written in standard relational notation format matching Joseph's submission:

```
1. Department (dept_id PK, dept_name) 2. Student (matric_no PK, name, dept_id FK → Depa rtment) 3. Lecturer (lecturer_id PK, lecturer_name, dept_id FK → Department) 4. Course (cours e_code PK, course_title) 5. Enrolment (matric_no PK/FK → Student, course_code PK/FK → Course, session) 6. CourseAndLecturer (course_code PK/FK → Course, lecturer_id PK/FK → Lecturer, sess ion PK)
```

PART 3 & 4: MYSQL WORKBENCH EER DIAGRAM & DELIVERABLES

MySQL Workbench Implementation Specifications

- **Schema Renaming:** Set physical schema name from `mydb` to `sen307_2411477` (using student matriculation details).
- **Table Configurations:**
  - Check PK and NN for primary keys (`dept_id`, `lecturer_id`, `matric_no`, `course_code`).
  - Set unique constraints on `matric_no` and `course_code`.
- **Relationship Setup:**
  - Use **1:n Non-Identifying Relationship** tool.
  - Click child table first (e.g., `Student`), then parent table (e.g., `Department`).
  - Verify foreign key column naming matches schema conventions (e.g., `dept_id`).

GitHub Command Sequence & Submission Verification

Execute the following terminal commands to commit and push all required assets to your repository:

```
# Navigate to lab repository cd sen307-labs # Create folder for Week 4 mkdir -p week04
# Move generated files into week04/ git add week04/ git commit -m "Week 4: ER diagram and nor malization to 3NF - Joseph Ijanu Skari (VUG/SEN/24/11477)" # Push to remote GitHub server git push origin main
```

Submission Checklist Verification

Deliverable File Path	Description	Status
`week04/lab2_erd.png`	Exported high-resolution EER Diagram image from Workbench	READY
`week04/lab2_normalization.pdf`	One-page/Full documentation showing 1NF, 2NF, and 3NF conversions	READY
`week04/lab2_model.mwb`	MySQL Workbench native design file (Required for Week 5 DDL)	READY
`GitHub Repository`	All week04 files pushed to GitHub remote repository	READY